TRANSFORMERS, FROM FIRST PRINCIPLES

Self-Attention

How tokens read and mix information

Ali Zindari

What is self-attention?



inside a Transformer layer

A crucial building block

Self-attention → Add & normalize → Feed-forward

Each output token keeps its position, but now carries information gathered from the prompt.

Imagine predicting the next word

Sun rises in the

input context

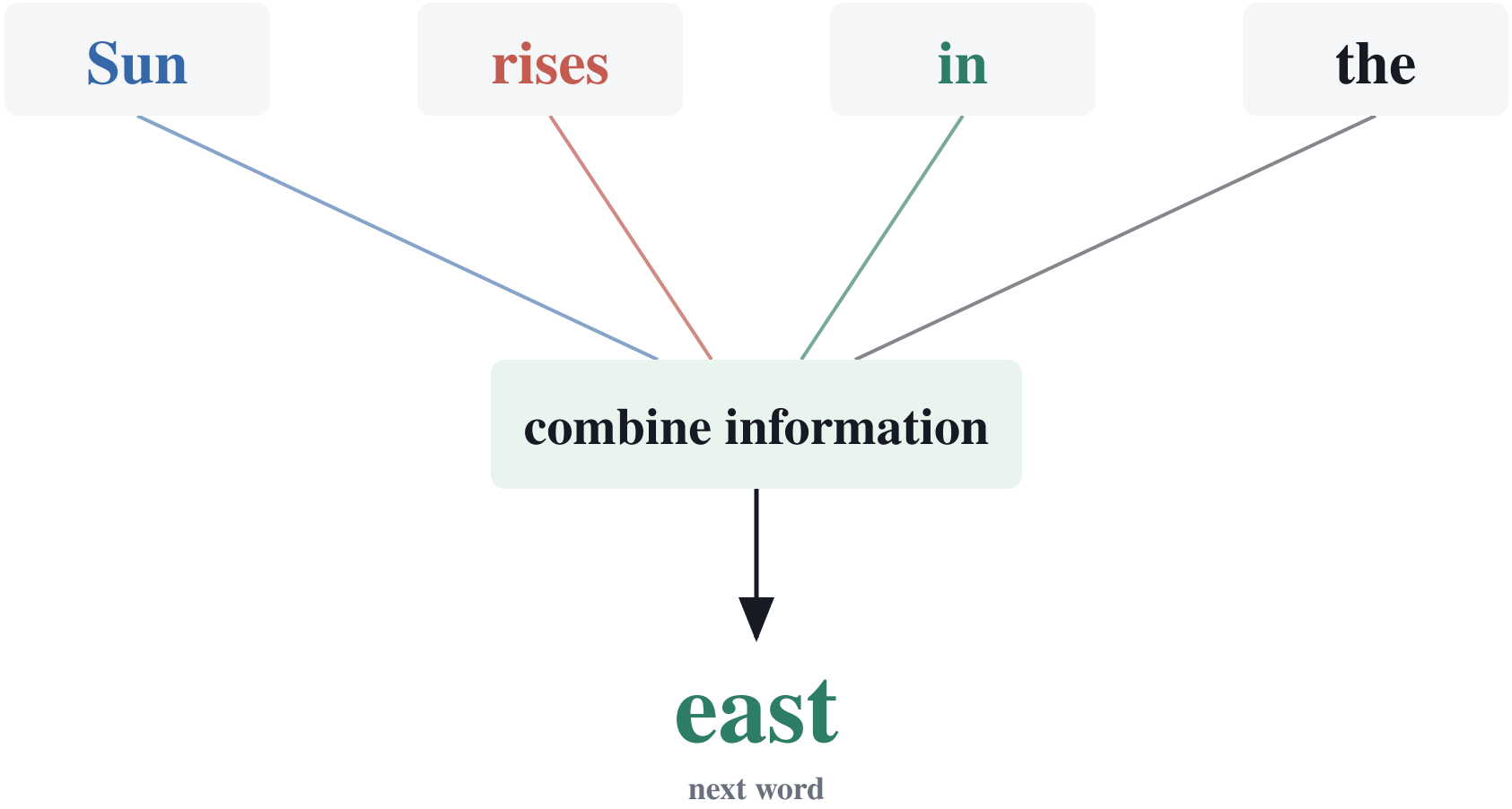
predict

?

east

target word

The answer depends on the whole context



Represent the input as token rows

simplified token-only example

$$\mathbf{X}$$

Sun	1	0	0	0
rises	0	1	0	0
in	0	0	1	0
the	0	0	0	1

one-hot rows in $\mathbb{R}^4$

one instance of

general form

$$\mathbf{X}$$

$$\begin{bmatrix} — \ \mathbf{x}_1^\top \ — \\ — \ \mathbf{x}_2^\top \ — \\ — \ \mathbf{x}_3^\top \ — \\ — \ \mathbf{x}_4^\top \ — \end{bmatrix}$$

token + position information

$\mathbf{x}_i^\top \in \mathbb{R}^{1\times d}$

Right multiplication transforms token features

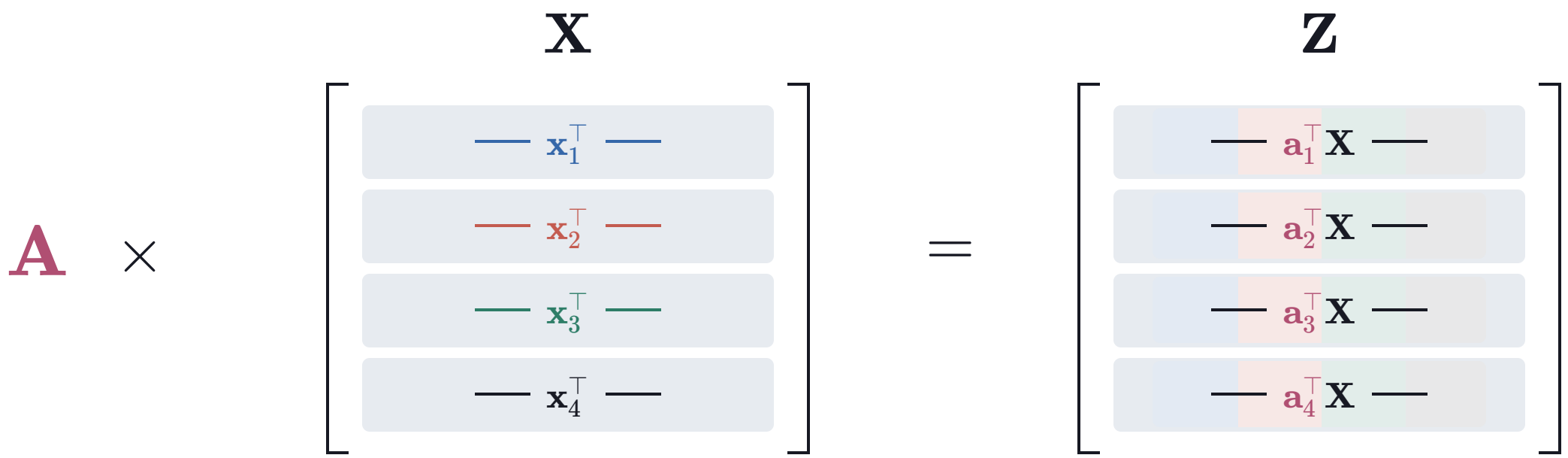
$$\mathbf{Z} = \mathbf{X}\mathbf{W}$$



the same $\mathbf{W}$ transforms the features of every token row

Left multiplication mixes token rows

$$\mathbf{Z} = \mathbf{A}\mathbf{X}$$



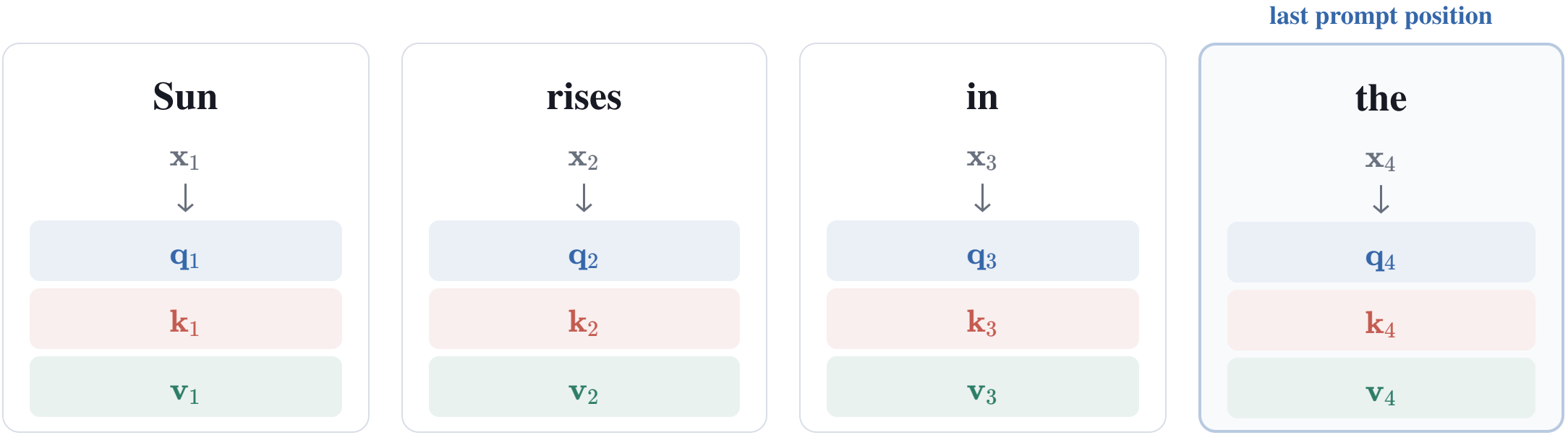
each row $\mathbf{a}_i^\top \mathbf{X}$ combines information from every input row

Right and left multiplication



Self-attention is just right and left multiplication.

Every prompt token creates $\mathbf{q}_i, \mathbf{k}_i, \mathbf{v}_i$



query — what this position reads

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q$$

key — how this position is matched

$$\mathbf{K} = \mathbf{X}\mathbf{W}_K$$

value — what this position can send

$$\mathbf{V} = \mathbf{X}\mathbf{W}_V$$

The prompt is not one query. Every prompt position produces all three vectors in parallel.

Each query can read only itself and the past

query ↓ · key →	k_1 Sun	k_2 rises	k_3 in	k_4 the
q_1 Sun	compare	future	future	future
q_2 rises	compare	compare	future	future
q_3 in	compare	compare	compare	future
q_4 the	compare	compare	compare	compare

$\mathbf{q}_i$ may compare with $\mathbf{k}_j$ only when $j \leq i$

visible this key may be compared

masked future information is unavailable

next

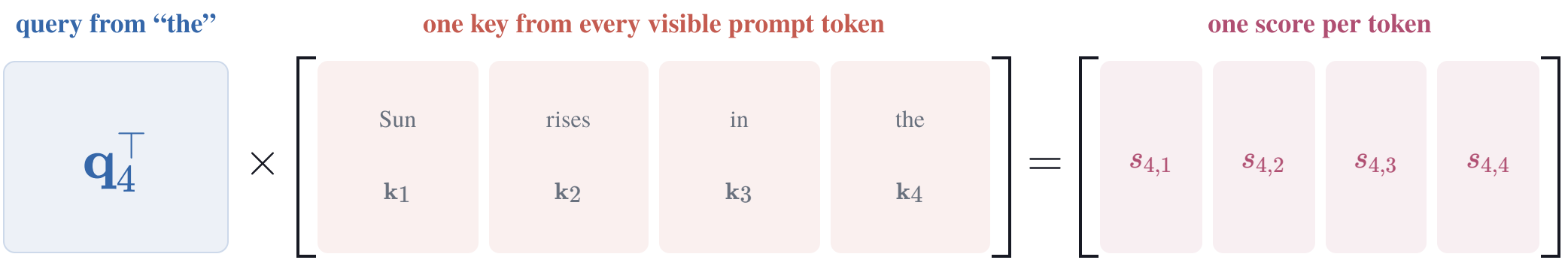
Zoom into the last row

$\mathbf{q}_4$ can compare with every key already in the prompt.

Compare the last query with every visible key

Sun **rises** **in** **the**

prompt · position 4 will predict the next token



$$s_{4,j} = \frac{\mathbf{q}_4^\top \mathbf{k}_j}{\sqrt{d_k}}, \qquad j \le 4$$

Each dot product is a learned **compatibility score**—not ordinary word correlation. Values are not used yet.

Softmax turns the scores into attention weights

$$\begin{bmatrix} s_{4,1} & s_{4,2} & s_{4,3} & s_{4,4} \end{bmatrix} \xrightarrow{\text{softmax}} \begin{bmatrix} a_{4,1} & a_{4,2} & a_{4,3} & a_{4,4} \end{bmatrix}$$

made-up weights to visualize the arithmetic

Sun	**rises**	**in**	**the**
from $s_{4,1}$	from $s_{4,2}$	from $s_{4,3}$	from $s_{4,4}$
$a_{4,1} = 0.34$	$a_{4,2} = 0.34$	$a_{4,3} = 0.12$	$a_{4,4} = 0.20$

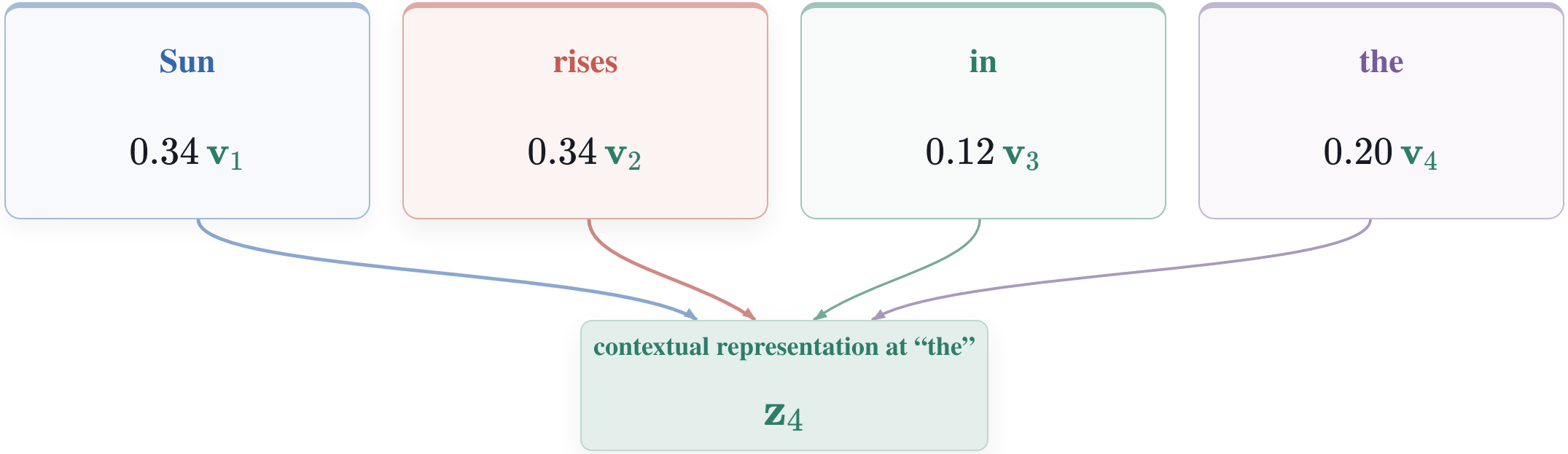
$$a_{4,1} + a_{4,2} + a_{4,3} + a_{4,4} = 1$$

One weight for each visible position; together they sum to 1. The values are still untouched.

The weights mix values from the prompt

$$\mathbf{z}_4 = \sum_{j \le 4} a_{4j} \mathbf{v}_j$$

the same weights now multiply the values



Important “east” is not in the prompt, so $\mathbf{v}_{\text{east}}$ does not exist yet.

Only afterward do we predict the next token

Sun **rises** **in** **the**

prompt

final contextual state

$\mathbf{z}_4$

at "the"

output projection →

vocabulary logits

$\mathbf{z}_4^\top \mathbf{W}_U$

one score per possible token

softmax →

next-token probabilities

east	0.72
west	0.06
sky	0.05
all others	0.17

$$p(x_5 \mid x_{1:4}) = \text{softmax}\left(\mathbf{z}_4^\top \mathbf{W}_U\right)$$

"east" first appears as a prediction—not as a key or value.

The left and right matrices in self-attention

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q \qquad \mathbf{K} = \mathbf{X}\mathbf{W}_K \qquad \mathbf{V} = \mathbf{X}\mathbf{W}_V$$

1 · match + mask

$$\mathbf{S} = \frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} + \mathbf{M}_{\text{causal}}$$

Compare queries with keys; hide future positions.

$\Longrightarrow$

2 · normalize

$$\mathbf{A} = \text{softmax}_{\text{row}}(\mathbf{S})$$

Turn each score row into attention weights.

$\Longrightarrow$

3 · retrieve

$$\mathbf{Z} = \mathbf{A}\mathbf{V}$$

Use those weights to mix the values.

$$\mathbf{Z} = \underbrace{\text{softmax}\left(\frac{(\mathbf{X}\mathbf{W}_Q)(\mathbf{X}\mathbf{W}_K)^\top}{\sqrt{d_k}} + \mathbf{M}_{\text{causal}}\right)}_{\text{left matrix } \mathbf{A}}\mathbf{X}\ \underbrace{\mathbf{W}_V}_{\text{right matrix}}$$

$\mathbf{A}$ mixes token rows; $\mathbf{W}_V$ mixes features within each token row.